



Encontro 4

Fluxo Colaborativo, Gitflow e Projeto Final

0:00–0:15

Revisão E3

0:15–1:00

Fork e Pull Request

1:00–1:30

Gitflow

1:30–2:45

Projeto em equipe

2:45–3:00

Apresentação final

```
git commit -m "Último encontro – bora colaborar!"
```

Revisão rápida — Encontro 3

Branches

Linhas de desenvolvimento paralelas. main sempre estável.

```
$ git checkout -b feature/nome
```

Merge e conflitos

Une branches. Conflito = mesma linha editada nos dois lados.
Resolver manualmente.

```
$ git merge nome
```

git fetch (seguro)

Baixa do remote sem aplicar. Inspecione antes de fazer merge.

```
$ git fetch
```

Ciclo de trabalho

O fluxo que você vai repetir todos os dias em uma empresa de tech.

```
$ pull → branch → commit → push
```



Fork — contribuindo para projetos que você não gerencia

Fork

Cópia do repositório para a SUA conta do GitHub. Você tem controle total. Pode editar, fazer push, abrir PR para o original.

Quando usar: projetos open source ou repos de outros usuários.

≠

Clone

Cópia local do repositório na sua máquina. Não cria um repositório na sua conta. Você precisa de permissão para fazer push.

Quando usar: projetos onde você é colaborador direto.

Fluxo completo:

fork (GitHub)



git clone do fork



git checkout -b feature/



git push



Pull Request

Manter o fork sincronizado com o repositório original

```
$ git remote add upstream <url-original>
```

— adicionar o repositório original como upstream

```
$ git fetch upstream
```

— baixar as atualizações do projeto original

```
$ git merge upstream/main
```

— aplicar as mudanças no seu fork

Pull Request — propondo e revisando mudanças

O PR é a forma padrão de colaboração em qualquer empresa de tecnologia

Anatomia de um bom PR

Título feat: adiciona validação de e-mail no cadastro

Descrição O que foi feito e por quê. Link para a issue relacionada.

Screenshots Print da tela ou GIF se for mudança visual.

Checklist ☐ Testado localmente ☐ Sem console.log

Revisor Marcar quem deve revisar o código.

Code Review — como revisar bem

Seja construtivo
● Critique o código, não a pessoa. "Sugiro usar map() aqui" em vez de "isso está errado".

Faça perguntas
● "Por que escolheu essa abordagem?" abre diálogo e pode revelar contexto importante.

Elogie boas soluções
● Code review não é só apontar problemas. Reconhecer o que está bom é importante.

Aprove ou solicite mudança
● Ao final: Approve (aprovado), Comment (comentário) ou Request changes (ajustes necessários).

Passo a passo — abrindo um Pull Request

1

Criar e trabalhar na feature branch

Tudo que fizemos no E3. Branch com nome descritivo, commits frequentes.

```
$ git checkout -b feature/meu-recurso
```

2

Enviar a branch para o GitHub

Publica a branch no repositório remoto.

```
$ git push -u origin feature/meu-recurso
```

3

Abrir o PR no GitHub (interface web)

GitHub sugere automaticamente abrir PR quando detecta um novo push. Clique em Compare & pull request.

4

Preencher título, descrição e revisor

Seguir a anatomia do slide anterior. Marcar um colega como revisor.

5

Reviewer aprova e faz o merge

Após aprovação, clicar em Merge pull request. Escolher Squash, Rebase ou Merge commit.

6

Deletar a branch após o merge

GitHub oferece o botão automaticamente. Manter o repositório limpo.

```
$ git branch -d feature/meu-recurso
```



Gitflow — por que precisamos de uma convenção?

Sem convenção — o caos

nova-tela

teste-joao

v2

fix

branch1

minha-branch-final

branch-prod-2

Com Gitflow — clareza imediata

● main

● develop

● feature/tela-cadastro

● feature/validacao-email

● release/v1.2.0

● hotfix/corrige-login

O que é o Gitflow?

Gitflow é um conjunto de convenções de nomenclatura e uso de branches. Não é uma ferramenta — é um acordo entre o time sobre como nomear e organizar o trabalho. Criado por Vincent Driessen (2010) e adotado pela maioria das empresas de tecnologia.

Gitflow — as branches e seus papéis

main	Sempre estável, vai para produção. Nunca se edita direto aqui. Recebe apenas de release/ e hotfix/.	permanente
develop	Integração contínua de features. Branch base para criar features. Sempre tem o código mais atualizado em desenvolvimento.	permanente
feature/nome	Criada a partir de develop. Uma branch por tarefa/card. Mergeada de volta para develop via PR. Deletada após o merge.	temporária
release/v1.0	Criada a partir de develop quando o código está pronto. Apenas correções de bugs. Mergeada para main E develop. Gera uma tag de versão.	temporária
hotfix/nome	Criada a partir de main. Correção urgente em produção. Mergeada para main E develop. Tem prioridade máxima.	temporária

Gitflow — fluxo geral





Projeto final — colaboração em equipe

Simulação real de um fluxo de desenvolvimento em equipe

Grupos de 3 a 4 alunos · 1 repositório por grupo · cada aluno é responsável por uma feature

1

Um aluno cria o repositório e adiciona os colegas

Settings → Collaborators → Add people. Todos recebem acesso de escrita.

2

Todos clonam o repositório

Cada aluno clona na sua máquina.

```
$ git clone https://github.com/grupo/projeto.git
```

3

Cada aluno cria sua feature branch

Usar convenção Gitflow: feature/nome-da-feature.

```
$ git checkout -b feature/minha-parte
```

4

Desenvolver e commitar

Implementar a parte atribuída. Commits frequentes e com mensagens claras.

```
$ git add . && git commit -m "feat: implementa X"
```

5

Push e abrir Pull Request

Enviar a branch e abrir PR para um colega revisar.

```
$ git push -u origin feature/minha-parte
```

6

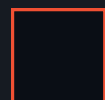
Revisar o PR do colega e mergear

Ler o código, deixar um comentário, aprovar e mergear na main.



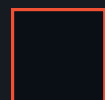
Apresentação dos projetos

2 minutos por grupo — mostrar o repositório no GitHub



Perfil do GitHub

Mostrar o seu perfil — você já tem ao menos 1 repositório público criado durante o curso.



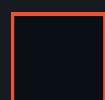
Histórico de commits

`git log --oneline`. Mostrar os commits com mensagens descritivas feitos durante o projeto.



Branches criadas

`git branch -a`. Mostrar as feature branches com nomenclatura Gitflow.



Pull Requests abertos/merged

Aba Pull Requests no GitHub. Mostrar os PRs com descrição e o code review do colega.



Conflito resolvido (bônus)

Se o grupo teve e resolveu um conflito, mostrar o commit de merge com os marcadores resolvidos.

Resumo dos comandos — Encontro 4

Fork e remote

`fork` (GitHub UI)

copia repo para sua conta

`git remote add upstream <url>`

adiciona repo original

`git fetch upstream`

sincronizar com original

`git merge upstream/main`

aplicar atualizações

Pull Request

`git push -u origin branch`

publica branch

PR (GitHub UI)

abrir pull request

Code Review (GitHub UI)

revisar e comentar

`git branch -d branch`

deletar após merge

Gitflow

main

sempre estável / produção

develop

integração de features

feature/nome

uma tarefa por branch

release/v1.0

preparação de lançamento

hotfix/nome

correção urgente em prod



ENCONTRO 4

FINALIZADO!



Parabéns por concluir toda a jornada!

Você aprendeu a **colaborar em equipe** e a trabalhar em **projetos do jeito profissional**.

O QUE VIMOS NO ENCONTRO 4:



FORK

Contribuindo para projetos que você não gerencia.



PULL REQUEST

Propondo e revisando mudanças



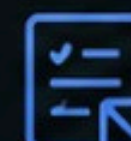
GITFLOW

Fluxo de trabalho profissional



PROJETO EM EQUIPE

Colaboração, comunicação e boas práticas



REVISÃO GERAL

Consolidação de tudo que aprendemos



VOCÊ CONCLUIU TODOS OS 4 ENCONTROS!

Agora você tem uma base sólida em Git e GitHub para continuar evoluindo e aplicando no mundo real. O importante é: **prática constante!**



Continue praticando, construindo e evoluindo!

O futuro é todo seu. Bora codar!

